Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Основы системного программирования»

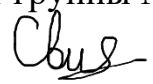
ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Работа с файлами и каталогами»

Вариант 19

Выполнил:

Свиржевский Алексей Денисович, студент группы N3252



(подпись)

Проверил:

Грозов Владимир Андреевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание 3
Задание 4
1. ОПИСАНИЕ ЛОГИКИ РАБОТЫ 5
2. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ 6
ВЫВОД 9
Приложение 10

ЗАДАНИЕ

Цель работы: изучение принципов создания и использования разделяемых (динамических) библиотек в ОС Linux, освоение работы с функциями dlopen, dlsym, dlclose, а также механизмы обработки длинных опций командной строки с помощью getopt_long.

Задание:

Разработать программу, выполняющую рекурсивный поиск файлов, где критерии поиска задаются через подключаемые динамические библиотеки (плагины).

Программа должна поддерживать стандартные опции объединения результатов (-A, -O), инверсии (-N) и явного указания каталога с плагинами (-P).

Задание по варианту плагина (Вариант 19):

Реализовать плагин, обрабатывающий опцию --ipv4-addr-bin <значение>.

Назначение: поиск файлов, содержащих заданное значение IPv4-адреса в бинарной (little-endian или big-endian) форме (4 последовательных октета).

1. ОПИСАНИЕ ЛОГИКИ РАБОТЫ

Архитектура решения разделена на программу и динамически подгружаемый плагин.

1. **Хост-программа (main.c):** выполняет обход аргументов командной строки в два прохода. На первом проходе (getopt) извлекается директория с плагинами (опция -P), после чего происходит их поиск и загрузка через dlopen. Хост динамически запрашивает поддерживаемые плагином опции (через plugin_get_info), объединяет их со своими стандартными опциями и на втором проходе (getopt_long) распределяет аргументы между плагинами. Обход файлов выполняется с помощью ftw.
2. **Плагин (libasdN3252.c):** принимает целевой IPv4 адрес в строковом виде, преобразует его в бинарный сетевой порядок (Big Endian) с помощью функции inet_pton, а затем формирует копию в Little Endian с помощью встроенных битовых операций (__builtin_bswap32). Чтение проверяемого файла осуществляется блоками по 4 КБ. Для предотвращения пропуска искомой последовательности на границе блоков реализован откат файлового указателя (fseek на -3 байта назад).
3. **Логика комбинирования:** хост-программа копит коды возврата от плагинов, объединяя их по правилам булевой алгебры (И/ИЛИ), после чего применяет операцию логического отрицания, если задан флаг -N. При возникновении критической ошибки при чтении файла (плагин возвращает < 0), файл игнорируется.

2. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

Сборка проекта и тестирование осуществляются с помощью утилиты make.

Содержимое Makefile:

```
SHELL := /bin/bash
CC = gcc
CFLAGS = -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE
TARGET = lab2asdN3252
PLUGIN = libasdN3252.so
all: $(TARGET) $(PLUGIN)
$(TARGET): main.c plugin_api.h
    $(CC) $(CFLAGS) -o $@ main.c -ldl
$(PLUGIN): libasdN3252.c plugin_api.h
    $(CC) $(CFLAGS) -shared -fPIC -o $@ libasdN3252.c
clean:
    chmod 777 test_dir/no_access.bin test_dir/no_access_dir 2>/dev/null || true
    rm -f $(TARGET) $(PLUGIN) valgrind.txt
    rm -rf test_dir
test: all
    @echo -e "\n"
    @echo " Шаг 1. ПОДГОТОВКА ПОЛИГОНА ДЛЯ ТЕСТИРОВАНИЯ "
     rm -rf test_dir && mkdir -p test_dir/sub_dir test_dir/custom_plugins
test_dir/no_access_dir
    # 192.168.8.1 в Big Endian (C0 A8 08 01)
    printf "\xC0\xA8\x08\x01\x99\x88" > test_dir/match_be.bin
    # 192.168.8.1 в Little Endian (01 08 A8 C0)
    printf "\x00\x01\x08\xA8\xC0\xFF" > test_dir/sub_dir/match_le.bin
    # Файл без адреса
    printf "\x11\x22\x33\x44" > test_dir/no_match.bin
    # Файлы с ошибками доступа
    echo "secret" > test_dir/no_access.bin
    chmod 000 test_dir/no_access.bin
    chmod 000 test_dir/no_access_dir
    @echo -e "\n"
    @echo " Шаг 2. ТЕСТ СТАНДАРТНЫХ ФЛАГОВ (-h и -v) "
```

```
	./$(TARGET) -v
	./$(TARGET) -h
	@echo -e "\n"
	@echo " Шаг 3. ТЕСТ ОШИБОК ПОЛЬЗОВАТЕЛЯ "
	@echo "[Ожидается ошибка: Неизвестная опция]"
	./$(TARGET) --unknown-opt test_dir || true
	@echo -e "\n[Ожидается ошибка: Некорректный IP-адрес]"
	LAB2DEBUG=1 ./$(TARGET) --ipv4-addr-bin 999.99.99.99 test_dir || true
	@echo -e "\n[Ожидается вывод справки: Нет каталога]"
	./$(TARGET) --ipv4-addr-bin 192.168.8.1 || true
	@echo -e "\n"
	@echo " Шаг 4. ПОИСК IP (Big & Little Endian) "
	@echo "Ожидаем найти: match_be.bin и match_le.bin"
	./$(TARGET) --ipv4-addr-bin 192.168.8.1 test_dir
	@echo -e "\n"
	@echo " Шаг 5. ИНВЕРСИЯ ПОИСКА (Опция -N) "
	  @echo "Ожидаем найти: no_match.bin (файлы без прав доступа должны
игнорироваться)"
	./$(TARGET) -N --ipv4-addr-bin 192.168.8.1 test_dir
	@echo -e "\n"
	@echo " Шаг 6. ИСПОЛЬЗОВАНИЕ КАСТОМНОЙ ДИРЕКТОРИИ (Опция -P) "
	cp $(PLUGIN) test_dir/custom_plugins/
	# Вызываем из другой папки, чтобы плагин не нашелся по умолчанию
	./$(TARGET) -P test_dir/custom_plugins --ipv4-addr-bin 192.168.8.1 test_dir
	@echo -e "\n"
	@echo " Шаг 7. VALGRIND (ПРОВЕРКА УТЕЧЕК ПАМЯТИ) "
	 valgrind --leak-check=full --show-leak-kinds=all --log-file=valgrind.txt
./$(TARGET) --ipv4-addr-bin 192.168.8.1 test_dir || true
	@grep "HEAP SUMMARY:" valgrind.txt -A 5 || true
	@echo -e "\n"
	@echo "ТЕСТИРОВАНИЕ ЗАВЕРШЕНО УСПЕШНО"
	chmod 777 test_dir/no_access.bin test_dir/no_access_dir || true
archive: clean
	cd .. && tar -czvf $(TARGET).tar.gz $(TARGET)/
```

make test (рис. 1 и 2):

```
 Шаг 1. ПОДГОТОВКА
rm -rf test_dir && mkdir -p test_dir/sub_dir test_dir/custom_plugins test_dir/no_access_dir
# 192.168.8.1 в Big Endian (C0 A8 08 01)
printf "\xC0\xA8\x08\x01\x99\x88" > test_dir/match_be.bin
# 192.168.8.1 в Little Endian (01 08 A8 C0)
printf "\x00\x01\x08\xA8\xC0\xFF" > test_dir/sub_dir/match_le.bin
# Файл без адреса
printf "\x11\x22\x33\x44" > test_dir/no_match.bin
# Файлы с ошибками доступа
echo "secret" > test_dir/no_access.bin
chmod 000 test_dir/no_access.bin
chmod 000 test_dir/no_access_dir


 Шаг 2. ТЕСТ СТАНДАРТНЫХ ФЛАГОВ (-h и -v)
./lab2asdN3252 -v
Версия: 2.0
Свиржевский А.Д., N3252
./lab2asdN3252 -h
Использование: ./lab2asdN3252 [опции] [каталог]
Стандартные опции:
  -P dir         Каталог с плагинами
  -A             Объединение условий по 'И' (по умолчанию)
  -O             Объединение условий по 'ИЛИ'
  -N             Инвертировать условие поиска
  -v, --version  Вывести версию
  -h, --help     Вывести справку

Опции загруженных плагинов:
--- Плагин libasdN3252.so (Поиск IPv4 адресов в бинарном виде, Свиржевский А.Д., N3252) ---
  --ipv4-addr-bin       Поиск файлов, содержащих заданное значение IPv4-адреса в бинарной форме (little-endian или big-endian)


 Шаг 3. ТЕСТ ОШИБОК ПОЛЬЗОВАТЕЛЯ
[Ожидается ошибка: Неизвестная опция]
./lab2asdN3252 --unknown-opt test_dir || true
./lab2asdN3252: unrecognized option '--unknown-opt'

[Ожидается ошибка: Некорректный IP-адрес]
LAB2DEBUG=1 ./lab2asdN3252 --ipv4-addr-bin 999.99.99.99 test_dir || true
[MAIN] Успешно загружен плагин: libasdN3252.so
[libasdN3252] Ошибка: некорректный IPv4 адрес '999.99.99.99'
[libasdN3252] Ошибка: некорректный IPv4 адрес '999.99.99.99'
[libasdN3252] Ошибка: некорректный IPv4 адрес '999.99.99.99'
[libasdN3252] Ошибка: некорректный IPv4 адрес '999.99.99.99'
[Ожидается вывод справки: Нет каталога]
./lab2asdN3252 --ipv4-addr-bin 192.168.8.1 || true
Использование: ./lab2asdN3252 [опции] [каталог]
Стандартные опции:
  -P dir         Каталог с плагинами
  -A             Объединение условий по 'И' (по умолчанию)
  -O             Объединение условий по 'ИЛИ'
  -N             Инвертировать условие поиска
  -v, --version  Вывести версию
  -h, --help     Вывести справку

Опции загруженных плагинов:
--- Плагин libasdN3252.so (Поиск IPv4 адресов в бинарном виде, Свиржевский А.Д., N3252) ---
  --ipv4-addr-bin       Поиск файлов, содержащих заданное значение IPv4-адреса в бинарной форме (little-endian или big-endian)


 Шаг 4. ПОИСК IP (Big & Little Endian)
Ожидаем найти: match_be.bin и match_le.bin
./lab2asdN3252 --ipv4-addr-bin 192.168.8.1 test_dir
test_dir/match_be.bin
test_dir/sub_dir/match_le.bin


 Шаг 5. ИНВЕРСИЯ ПОИСКА (Опция -N)
Ожидаем найти: no_match.bin (файлы без прав доступа должны игнорироваться)
./lab2asdN3252 -N --ipv4-addr-bin 192.168.8.1 test_dir
test_dir/no_access.bin
test_dir/no_match.bin


 Шаг 6. ИСПОЛЬЗОВАНИЕ КАСТОМНОЙ ДИРЕКТОРИИ (Опция -P)
cp libasdN3252.so test_dir/custom_plugins/
# Вызываем из другой папки, чтобы плагин не нашелся по умолчанию
./lab2asdN3252 -P test_dir/custom_plugins --ipv4-addr-bin 192.168.8.1 test_dir
test_dir/match_be.bin
test_dir/sub_dir/match_le.bin
```

Рисунок 1

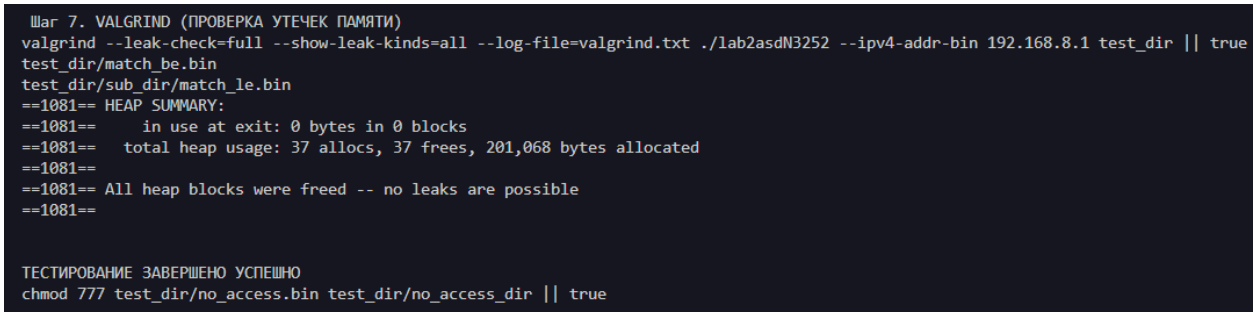


Рисунок 2

ВЫВОД

В ходе выполнения лабораторной работы была успешно спроектирована и реализована расширяемая архитектура приложения на базе динамически загружаемых плагинов. Освоены навыки работы с API динамической загрузки (dlfcn.h). Реализована логика двухпроходного разбора аргументов командной строки. В плагине реализован оптимизированный алгоритм поиска бинарных данных, учитывающий особенности порядка байт и блочного чтения из файлового потока. Успешное прохождение тестов Valgrind подтвердило корректное управление памятью, выделяемой под структуры плагинов и динамические массивы опций getopt_long.

ПРИЛОЖЕНИЕ

Исходный код программы main.c:

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
#include <errno.h>
#include <ftw.h>
#include <dlfcn.h>
#include <getopt.h>
#include <dirent.h>
#include <unistd.h>
#include <limits.h>
#include "plugin_api.h"

// Глобальные переменные для управления плагинами и логикой объединения условий
typedef enum { OP_AND, OP_OR } LogicOp;

// Структура для хранения информации о загруженных плагинах и их состоянии
typedef struct {
    void *handle;
    char *name;
    struct plugin_info info;
    int (*get_info)(struct plugin_info*);
    int (*process_file)(const char*, struct option[], size_t);

    struct option *active_opts;
    size_t active_opts_len;
    size_t active_opts_cap;
```

```
} Plugin;

Plugin *plugins = NULL;
size_t plugins_count = 0;

LogicOp global_op = OP_AND;
bool invert_result = false;
bool debug_mode = false;

int *opt_to_plugin_map = NULL;

// Функция для печати справки
void print_help(const char *prog_name) {
    printf("Использование: %s [опции] [каталог]\n", prog_name);
    printf("Стандартные опции:\n");
    printf("  -P dir         Каталог с плагинами\n");
      printf("   -A                  Объединение условий по 'И' (по
умолчанию)\n");
    printf("  -O             Объединение условий по 'ИЛИ'\n");
    printf("  -N             Инвертировать условие поиска\n");
    printf("  -v, --version  Вывести версию\n");
    printf("  -h, --help     Вывести справку\n\n");

    printf("Опции загруженных плагинов:\n");
    for (size_t i = 0; i < plugins_count; i++) {
           printf("--- Плагин %s (%s, %s) ---\n", plugins[i].name,
plugins[i].info.plugin_purpose, plugins[i].info.plugin_author);
        for (size_t j = 0; j < plugins[i].info.sup_opts_len; j++) {
           printf("  --%s\t%s\n", plugins[i].info.sup_opts[j].opt.name,
plugins[i].info.sup_opts[j].opt_descr);
        }
    }
}
```

```
// Функция для загрузки плагинов из указанного каталога
void load_plugins(const char *dir_path) {
    DIR *d = opendir(dir_path);
    if (!d) {
        if (debug_mode) fprintf(stderr, "[MAIN] Не удалось открыть папку
плагинов: %s\n", dir_path);
        return;
    }

    struct dirent *dir;
    while ((dir = readdir(d)) != NULL) {
        size_t len = strlen(dir->d_name);
        if (len > 3 && strcmp(dir->d_name + len - 3, ".so") == 0) {
            char path[PATH_MAX];
           snprintf(path, sizeof(path), "%s/%s", dir_path, dir->d_name);

            void *handle = dlopen(path, RTLD_LAZY);
            if (!handle) {
                if (debug_mode) fprintf(stderr, "[MAIN] Ошибка загрузки
%s: %s\n", path, dlerror());
                continue;
            }

            Plugin p = {0};
            p.handle = handle;
            p.name = strdup(dir->d_name);
            p.get_info = dlsym(handle, "plugin_get_info");
            p.process_file = dlsym(handle, "plugin_process_file");

            if (!p.get_info || !p.process_file || p.get_info(&p.info) <
0) {
```

```
                if (debug_mode) fprintf(stderr, "[MAIN] Неверный API в
плагине %s\n", path);
                dlclose(handle);
                free(p.name);
                continue;
            }

            p.active_opts_cap = 4;
              p.active_opts = malloc(p.active_opts_cap * sizeof(struct
option));
            p.active_opts_len = 0;

                 plugins = realloc(plugins, (plugins_count + 1) *
sizeof(Plugin));
            plugins[plugins_count++] = p;
              if (debug_mode) fprintf(stderr, "[MAIN] Успешно загружен
плагин: %s\n", p.name);
        }
    }
    closedir(d);
}

int ftw_cb(const char *fpath, const struct stat *sb, int typeflag) {
    (void)sb;

    if (typeflag == FTW_DNR) {
          if (debug_mode) fprintf(stderr, "[MAIN] Ошибка доступа к
каталогу: %s\n", fpath);
        return 0;
    }

    if (typeflag != FTW_F) return 0;
```

```
    bool final_res = (global_op == OP_AND) ? true : false;
    bool plugins_ran = false;
    bool plugin_error = false;

    for (size_t i = 0; i < plugins_count; i++) {
        if (plugins[i].active_opts_len > 0) {
            plugins_ran = true;
                        int   res   =   plugins[i].process_file(fpath,
plugins[i].active_opts, plugins[i].active_opts_len);

            if (res < 0) {
                plugin_error = true;
                break;
            }

            bool match = (res == 0);

            if (global_op == OP_AND) final_res = final_res && match;
            else final_res = final_res || match;
        }
    }

    if (!plugins_ran || plugin_error) return 0;

    if (invert_result) final_res = !final_res;

    if (final_res) {
        printf("%s\n", fpath);
    }
    return 0;
}

// Функция для получения директории исполняемого файла
```

```
char* get_exe_dir() {
    static char buf[PATH_MAX];
    ssize_t len = readlink("/proc/self/exe", buf, sizeof(buf) - 1);
    if (len != -1) {
        buf[len] = '\0';
        char *last_slash = strrchr(buf, '/');
        if (last_slash) *last_slash = '\0';
        return buf;
    }
    return ".";
}

int main(int argc, char *argv[]) {
    debug_mode = (getenv("LAB2DEBUG") != NULL);

    char *plugin_dir = get_exe_dir();
    int opt;
    opterr = 0;
    while ((opt = getopt(argc, argv, "P:AONvh")) != -1) {
        if (opt == 'P') plugin_dir = optarg;
    }

    load_plugins(plugin_dir);

    size_t total_long_opts = 2;
    for (size_t i = 0; i < plugins_count; i++) {
        total_long_opts += plugins[i].info.sup_opts_len;
    }

    struct option *long_opts = calloc(total_long_opts + 1, sizeof(struct
option));
    opt_to_plugin_map = calloc(total_long_opts + 1000, sizeof(int));
```

```
    long_opts[0] = (struct option){"help", no_argument, 0, 'h'};
    long_opts[1] = (struct option){"version", no_argument, 0, 'v'};

    size_t opt_idx = 2;
    int current_val = 1000;

    for (size_t i = 0; i < plugins_count; i++) {
        for (size_t j = 0; j < plugins[i].info.sup_opts_len; j++) {
            struct option o = plugins[i].info.sup_opts[j].opt;
            o.val = current_val;
            long_opts[opt_idx++] = o;
            opt_to_plugin_map[current_val - 1000] = i;
            current_val++;
        }
    }

    optind = 1;
    opterr = 1;
    int longindex = 0;

      while ((opt = getopt_long(argc, argv, "P:AONvh", long_opts,
&longindex)) != -1) {
        if (opt == 'h') {
            print_help(argv[0]);
            return 0;
        } else if (opt == 'v') {
            printf("Версия: 2.0\nСвиржевский А.Д., N3252\n");
            return 0;
        } else if (opt == 'A') {
            global_op = OP_AND;
        } else if (opt == 'O') {
            global_op = OP_OR;
        } else if (opt == 'N') {
```

```
            invert_result = true;
        } else if (opt == 'P') {
            // Уже обработали
        } else if (opt >= 1000) {
            int p_idx = opt_to_plugin_map[opt - 1000];
            Plugin *p = &plugins[p_idx];

            if (p->active_opts_len == p->active_opts_cap) {
                p->active_opts_cap *= 2;
                        p->active_opts  =  realloc(p->active_opts,  p-
>active_opts_cap * sizeof(struct option));
            }

            struct option active = {0};
            active.name = long_opts[longindex].name;
            active.flag = (int *)optarg;

            p->active_opts[p->active_opts_len++] = active;
        } else {
            return 1;
        }
    }

    const char *target_dir = ".";
    if (optind < argc) {
        target_dir = argv[optind];
    } else {
        print_help(argv[0]);
        return 0;
    }

    if (ftw(target_dir, ftw_cb, 20) == -1) {
        perror("ftw");
```

```
    }

    for (size_t i = 0; i < plugins_count; i++) {
        free(plugins[i].name);
        free(plugins[i].active_opts);
        dlclose(plugins[i].handle);
    }
    free(plugins);
    free(long_opts);
    free(opt_to_plugin_map);

    return 0;
}
```

Исходный код плагина libasdN3252.c:

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <arpa/inet.h>
#include <stdbool.h>
#include "plugin_api.h"

// Описание опции, которую предоставляет наш плагин
static struct plugin_option my_options[] = {
    {
        {"ipv4-addr-bin", required_argument, 0, 0},
          "Поиск файлов, содержащих заданное значение IPv4-адреса в
бинарной форме (little-endian или big-endian)"
    }
```

```
};

int plugin_get_info(struct plugin_info* ppi) {
    if (!ppi) return -1;
    ppi->plugin_purpose = "Поиск IPv4 адресов в бинарном виде";
    ppi->plugin_author = "Свиржевский А.Д., N3252";
    ppi->sup_opts_len = 1;
    ppi->sup_opts = my_options;
    return 0;
}

// Целевая функция: проверяем конкретный файл
int plugin_process_file(const char *fname, struct option in_opts[],
size_t in_opts_len) {
    char *ip_str = NULL;
    bool debug = (getenv("LAB2DEBUG") != NULL);

    // 1. Ищем нашу опцию среди тех, что передала основная программа
    for (size_t i = 0; i < in_opts_len; i++) {
        if (strcmp(in_opts[i].name, "ipv4-addr-bin") == 0) {
            ip_str = (char *)in_opts[i].flag; // По условию API аргумент
лежит в поле flag
            break;
        }
    }

    if (!ip_str) {
        return 1; // Опция не передана, файл не подходит
    }

    // 2. Парсим IPv4 адрес в бинарный вид (Big Endian)
    uint32_t ip_be;
    if (inet_pton(AF_INET, ip_str, &ip_be) != 1) {
```

```
        if (debug) fprintf(stderr, "[libasdN3252] Ошибка: некорректный
IPv4 адрес '%s'\n", ip_str);
        errno = EINVAL;
        return -1;
    }

    // Делаем копию для Little Endian
    uint32_t ip_le = __builtin_bswap32(ip_be);

    // 3. Открываем файл и ищем байты
    FILE *f = fopen(fname, "rb");
    if (!f) {
        if (debug) fprintf(stderr, "[libasdN3252] Ошибка открытия '%s':
%s\n", fname, strerror(errno));
        return -1; // Ошибка доступа
    }

    uint8_t buf[4096];
    size_t bytes_read;
    off_t offset = 0;

    while ((bytes_read = fread(buf, 1, sizeof(buf), f)) > 0) {
        // Ищем 4 байта, поэтому идем до bytes_read - 3
        for (size_t i = 0; i + 3 < bytes_read; i++) {
              if (memcmp(buf + i, &ip_be, 4) == 0 || memcmp(buf + i,
&ip_le, 4) == 0) {
                if (debug) {
                    fprintf(stderr, "[libasdN3252] Найдено совпадение в
'%s' по смещению 0x%lX\n",
                            fname, (long)(offset + i));
                }
                fclose(f);
                return 0; // Файл соответствует критерию
```

```
            }
        }
        // Чтобы не пропустить адрес на стыке блоков, откатываемся на 3
байта назад
        if (bytes_read == sizeof(buf)) {
            fseek(f, -3, SEEK_CUR);
            offset += bytes_read - 3;
        } else {
            offset += bytes_read;
        }
    }

    if (ferror(f)) {
         if (debug) fprintf(stderr, "[libasdN3252] Ошибка чтения '%s':
%s\n", fname, strerror(errno));
        fclose(f);
        return -1;
    }

    fclose(f);
    return 1; // Не нашли
}
```